AI Smart College Assistant — Project Documentation

Author/Owner: Project Development Team
Technology Stack: Java 21, Spring Boot, React.js, LangChain4j, Gemini API, MySQL, Vector DB

1. Executive Summary & Abstract

The **AI Smart College Assistant** is an intelligent, web-based academic management platform designed to streamline student, faculty, and administrative workflows using modern Artificial Intelligence. By integrating **AI Agents** and **Retrieval-Augmented Generation (RAG)**, the system enables students to query official college documents, track attendance, submit assignments, generate customized study plans, practice mock interviews, and receive proactive reminders. Rather than relying on generic AI models, the application strictly indexes uploaded institutional documents to provide context-aware, highly accurate responses.

2. Problem Statement & Proposed Solution

2.1 Problem Statement

Students and faculty routinely face fragmented communication channels where critical academic information—such as exam schedules, lecture notes, syllabus updates, and placement drives—is scattered across PDFs, physical notice boards, emails, and messaging groups. This fragmentation leads to missed deadlines, inefficient study preparation, and a lack of personalized placement guidance.

2.2 Proposed Solution

An integrated end-to-end platform featuring:

- Centralized indexing and RAG-based search over institutional documents.
- Automated academic management (Attendance, Assignments, Timetable).
- Self-paced placement preparation (AI Mock Interviews, Aptitude & Coding Quizzes).
- Proactive notifications via email and calendar sync.

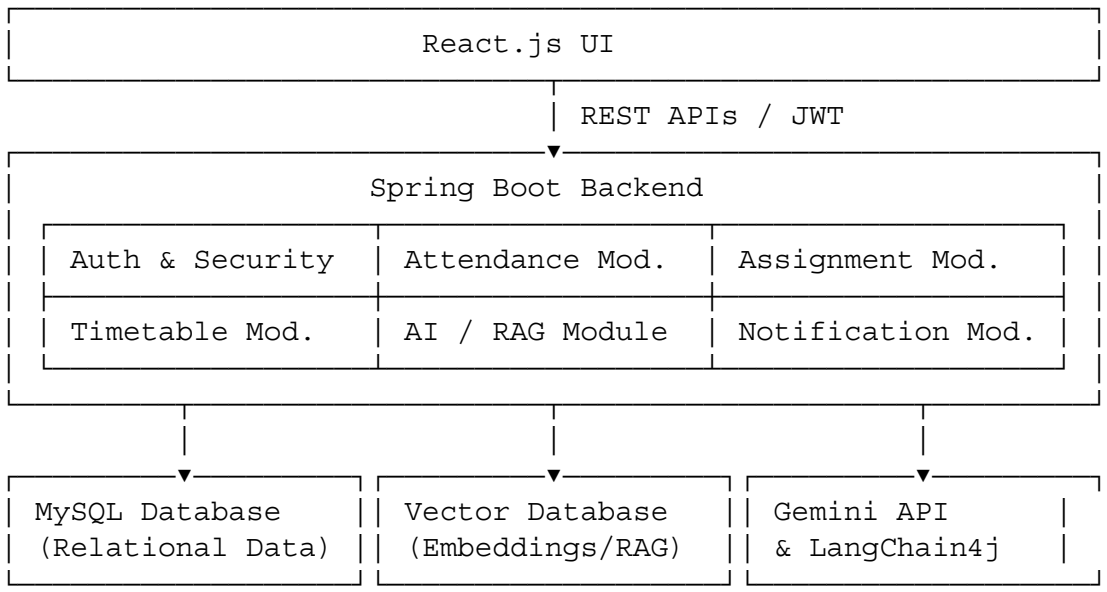
3. Key Objectives

- Centralize all academic resources and institutional notices in one secure portal.
- Provide instantaneous, document-backed answers using RAG technology.
- Enhance placement readiness through AI-driven interview practice and dynamic quiz generation.
- Automate administrative tracking for faculty and administrative bodies.

4. System Architecture & Component Interactions

The system is built on a decoupled, microservices-friendly architecture comprising a React.js single-page application, a Spring Boot REST API backend, and dedicated AI/Vector processing layers.

4.1 High-Level Architecture Diagram



5. Scope of Analysis

Current Scope (Phase 1–4)	Future Scope (Post-Release)
Student, Faculty, Admin Dashboards	Native Mobile Application (Android / iOS)
RAG-based AI Document Chatbot	Voice Assistant Integration
Attendance & Assignment Tracking	Optical Character Recognition (OCR) for Handwritten Notes
Placement & AI Mock Interview Prep	Facial Recognition for Attendance
Dynamic Quiz Generator	Multi-language & Learning Analytics Support

6. Detailed System Modules

6.1 Student Module

- **Dashboard:** Overview of upcoming assignments, attendance alerts, and class schedules.
- **AI Academic Assistant:** Ask questions directly to uploaded course notes and college notices.
- **Placement Suite:** Interactive mock interviews with AI feedback and specialized quiz

generation.

6.2 Faculty Module

- **Content Upload:** Distribute lecture slides, syllabus PDFs, and assignments.
- **Attendance Management:** Log daily student attendance and flag low-attendance warnings.
- **Notice Management:** Publish announcements with immediate notification dispatch.

6.3 Admin Module

- **User Management:** Role-based access assignment (Admin, Faculty, Student, HOD).
- **System Analytics:** Generate academic reports, course enrollments, and audit logs.

6.4 AI & RAG Engine Module

- **Document Ingestion:** Text extraction via Apache PDFBox, chunking, and embedding generation.
- **Vector Search:** Semantic similarity search for context extraction.
- **Response Generation:** Context injection into Gemini LLM using LangChain4j.

7. RAG (Retrieval-Augmented Generation) Workflow

1. **Document Ingestion:** Faculty uploads lecture notes/notices in PDF format.
2. **Text Processing:** Apache PDFBox extracts raw text, which is broken into semantic chunks.
3. **Embedding Storage:** Text chunks are transformed into vector embeddings and stored in the Vector Database.
4. **User Querying:** Student enters a question in the AI Chat window.
5. **Similarity Search:** Top-matching document chunks are retrieved from the Vector Database.
6. **Contextual Prompting:** The retrieved chunks are passed along with the query to the Gemini API.
7. **Answer Delivery:** An accurate, source-backed response is returned to the user interface.

8. Technical Specifications & Dependencies

Layer	Technology / Library	Purpose
Frontend	React.js, HTML5, CSS3, Bootstrap / Tailwind	Responsive, reactive user interface
Backend Core	Java 21, Spring Boot 3.x, Spring Data JPA	Core business logic, REST API framework
Security	Spring Security, JWT, BCrypt	Token-based authentication & RBAC
AI Framework	LangChain4j, Gemini API	LLM interaction, agent orchestration

Layer	Technology / Library	Purpose
PDF Processing	Apache PDFBox	Extraction of structured text from uploaded files
Databases	MySQL (Relational), Vector DB (Embeddings)	Data persistence and vector search
Containerization	Docker	Containerized deployment across environments

9. Integration APIs

- **Gemini API:** Drives conversational AI, summarization, and interview evaluation.
- **Google Calendar API:** Automatically syncs assignment submission deadlines and exam dates.
- **Gmail API:** Dispatches system notifications, login alerts, and weekly progress summaries.
- **OpenWeather API:** Provides live campus weather updates on the main dashboard.
- **Google Maps API:** Aids campus navigation for new students and visitors.
- **GitHub API:** Fetches student repositories for placement profile assessments.

10. Database Schema (Entities & Relationships)

The system uses the following primary tables in MySQL:

- Users / Students / Faculty — Core user profiles and credentials.
- Courses / Subjects — Academic mappings and curriculum data.
- Attendance — Daily attendance logs per subject.
- Assignments — Submission links, deadlines, and grade tracking.
- Timetable — Weekly schedule grids.
- Notes — Metadata for uploaded files and vector reference IDs.
- Quiz / Interview / AI_Chat_History — Stores user practice attempts and chat logs.

11. Project Roadmap & Development Phases

Phase	Key Deliverables
Phase 1: Foundation	Database schema setup, Spring Security + JWT authentication, Student/Faculty/Admin modules.
Phase 2: Core Academic Features	Attendance logging, assignment submission workflow, timetable scheduling, notification engine.
Phase 3: AI Engine & RAG	PDFBox text extraction, Vector DB integration, LangChain4j + Gemini API setup, Document Chatbot.
Phase 4: Advanced AI Features	Dynamic Quiz Generator, AI Mock Interview evaluator, Placement preparation module.
Phase 5: Quality Assurance	Unit & Integration testing, UAT, Docker

Phase	Key Deliverables
	containerization, final cloud deployment (Railway/Render/AWS).

12. Recommended Repository Structure

```
AI-Smart-College-Assistant/
├── frontend/               # React.js web portal
├── backend/                # Spring Boot REST API application
├── ai-service/             # LangChain4j & RAG orchestration logic
├── pdf-processing/         # Apache PDFBox extraction handlers
├── vector-db/              # Embeddings configuration & setup scripts
├── database/               # MySQL migrations and seed scripts
├── datasets/               # Sample interview & aptitude question
sets
├── docs/                   # System documentation & API
specifications
├── docker/                 # Dockerfile and Docker Compose configs
├── tests/                  # Unit, Integration, and UAT test suites
└── README.md
```

13. Non-Functional & Security Requirements

- **Stateless Authentication:** Secure session management via signed JWTs with role validation.
- **Data Protection:** Passwords hashed using BCrypt; HTTPS forced across all endpoints.
- **Performance:** Optimized vector indexing ensuring document search queries respond in < 2 seconds.
- **High Availability:** Scalable micro-architecture deployed via Docker containers.